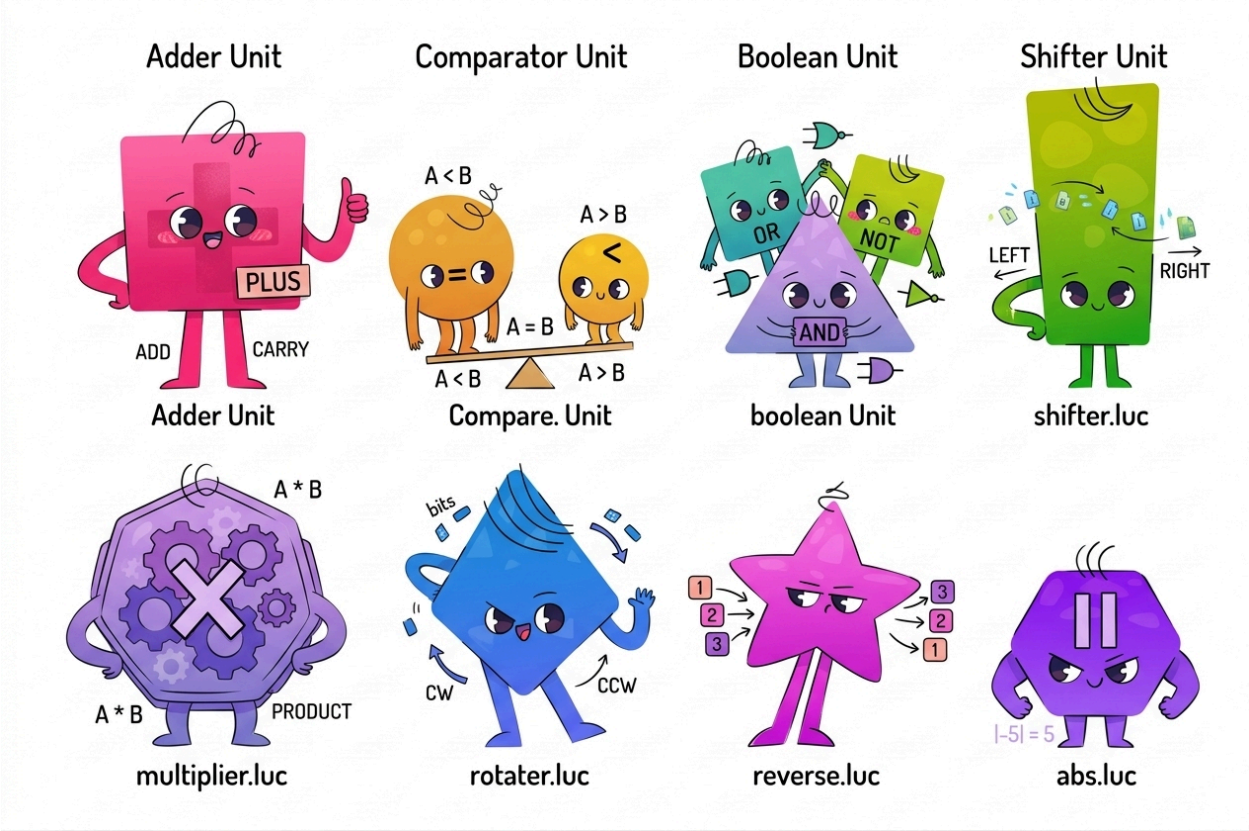
Appendix

1. ALU Design and Tests

Functionality of the ALU is broken up into 8 main units:

- Adder unit (adder.luc)
- Comparator unit (compare.luc)
- Boolean unit (boolean.luc)
- Shifter unit (shifter.luc)
- Multiplier unit (multiplier.luc)
- Rotator unit (rotater.luc)
- Reverse unit (reverse.luc)
- Absolute unit (abs.luc)



Inputs are fed into every single unit simultaneously, and the output of one of the units is selected via multiplexers. The multiplexers use ALUFN bits 5 to 1.
It must be noted that the output of the comparator unit depends on the inputs fed to the adder unit.

Adder unit

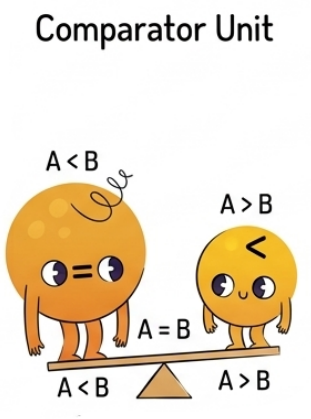
```
module adder #(
    SIZE ~ 32 : SIZE > 1
)(
    input a[SIZE],
    input b[SIZE],
    input alufn0,
    output s[SIZE],
    output z,
    output v,
    output n
) {
    sig xb[SIZE]
    rca rca(#SIZE(SIZE), .a(a), .b(xb), .cin(alufn0))
    always {
        xb = b ^ SIZEx{alufn0}
        s = rca.s
        z = ~| rca.s
        v = ( a[SIZE-1] & xb[SIZE-1] & ~rca.s[SIZE-1] ) | ( ~a[SIZE-1] & ~xb[SIZE-1] & rca.s[SIZE-1
        n = rca.s[SIZE-1]
    }
}
```

- 2 variable-bit numerical inputs (A and B)
- ALUFN bit 0: ‘1’ changes the operation to A - B
- 1 variable-bit numerical output (S)
- ‘Z’ output: Zero flag. Is 1 when all bits of S are 0.
- ‘V’ output: Overflow/Underflow flag, active when the output of adding two positive numbers is negative or when adding two negative numbers is positive. This is checked in practice by looking at the MSB of A, the MSB of XB as well as the MSB of S and then inspecting if the sign of the result differs from the operands.
- ‘N’ output: Negative/Sign flag. This is checked in practice by looking at the MSB of S. If it is 1, the result is negative and thus N is 1, flagging the result as negative.

Comparator unit

```
module compare (
    input z,    // ZVN wait for adder unit
    input v,
    input n,
    input alufn[6],
    output cmp
) {
    mux4to1 mux
    always {
        mux.data[0] = 0
        mux.data[1] = z
        mux.data[2] = n^v
        mux.data[3] = (n^v) | z
        mux.s0 = alufn[1]
        mux.s1 = alufn[2]
        cmp = mux.y
    }
}
```

- Inputs ‘Z’, ‘V’, and ‘N’ from the adder unit
- Takes in entire ALUFN bus for simplifying the interface, but actually only reads bit 1 and 2 to determine the comparison operation
- ALUFN bits used to control multiplexer for selecting comparison operator
- Output: 1 bit comparison operator
- ‘<’ operator implemented by Sign XOR Overflow/Underflow
    - A - B is negative (N flag is 1), and no overflow occurred (V is 0): A < B will be 1 and output bit is 1
    - A - B is positive (N flag is 0), and underflow occurred (V is 1): A < B
- ‘<=’ operator implemented by adding Z flag and OR with SIGN XOR
- A MUX 4-to-1 is used, with the 0th data being a zero signal, the 1st being the ‘Z’, 2nd being ‘N’ XOR ‘V’ and 3rd being (‘N’ XOR ‘V’) OR ‘Z’.

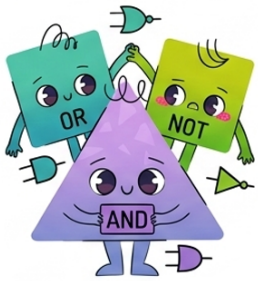


Boolean unit

```
module boolean (
    input a[32],
    input b[32],
    input alufn[6],
    output bool[32]
) {
    mux4to1 mux_4_32[32]
    always {
        mux_4_32.data = 32x{{alufn[3:0]}}
        mux_4_32.s0 = a
        mux_4_32.s1 = b
        bool = mux_4_32.y
    }
}
```

- 2 32-bit inputs ‘A’ and ‘B’ used to control 32 4-1 multiplexers
- Lower 4 bits of ALUFN used to drive the multiplexers
- Effectively a programmable truth table
- For example, if ‘AND’ is chosen as the boolean operator for 32-bit inputs ‘A’ and ‘B’, then ALUFN driven will be 1000, which is hardcoded such that the result of ‘A’[i] and ‘B’[i] is 1 only when both ‘A’[i] and ‘B’[i] is 1 (this results in the 4-1 MUX choosing data[3] which is 1).

Boolean Unit



Shifter unit

```
module x_bit_left_shifter #(
    SHIFT = 8 : SHIFT > -1 & SHIFT < 32,
    SIZE = 32 : SIZE > 0
)(
    input a[SIZE],
    input do_shift,
    input pad,
    output out[SIZE]
) {
    mux2 shift_unit[SIZE]
    sig in_lshift_unit[SIZE]

    always {
        shift_unit.s0 = SIZEx{do_shift}
        in_lshift_unit = c{a[SIZE-1-SHIFT:0], SHIFTx{pad}}

        repeat(i, SIZE) {
            shift_unit.in[i] = c{in_lshift_unit[i], a[i]}
        }
        out = shift_unit.out
    }
}
```

Shifter unit

```
module shifter (
    input a[32],
    input b[5],
    input alufn[6],
    output shift[32]
) {
    x_bit_left_shifter shifter[5](
        #SHIFT({8d16, 8d8, 8d4, 8d2, 8d1})
    )
    reverse r_in
    reverse r_out
    sig pad
    always {
        r_in.in = a
        pad = 0
        if (&alufn[1:0]) { // if both bits are 1, then it is a shift right with sign extension
            pad = a[-1] // take the MSB of A as the padding value
        }
        shifter.pad = 5x{pad}
        shifter.a[4] = a
        if (alufn[0]) {
            shifter.a[4] = r_in.out
        }
        shifter.do_shift[4] = b[4]

        repeat(i, 4) {
            shifter.a[i] = shifter.out[i+1]
            shifter.do_shift[i] = b[i]
        }
        shift = shifter.out[0]
        r_out.in = shifter.out[0]
        if (alufn[0]) {
            shift = r_out.out
        }
    }
}
```

Shifter unit

```
module reverse #(
    SIZE = 32: SIZE > 0
)(
    input in[SIZE],
    output out[SIZE]
) {
    always {
        repeat(i, 32) {
            out[i] = in[31-i]
        }
    }
}
```

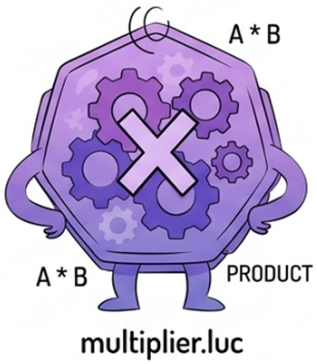
- 32-bit input ‘A’
- 5-bit input ‘B’: Chosen as a shift of 5b11111 is 32 - which will erase the original input and no higher number of shifts needs to be handled
- Takes in entire ALUFN bus for simplifying the interface, but actually only reads bit 1 and 0. A 0 in ALUFN[0] will signify a left shift operation while a 1 will signify a right shift operation. Then, if ALUFN[1] is 1, then it will signify a shift right with sign extension.
- Compact shifter used to save gates by reversing the bits on the input and then the output for right shift operations
- Reversing operation implemented as a separate module

Multiplier unit

```
        // bit 1 out
        mul[1] = fa.s[previous_row_fa_index]

        // do this for the 2nd row, 3rd row, ..., all the way to the 30th row (i.e. the 2nd last ro
        repeat (i, 29, 2){ // i == 2 to 30 (inclusive)

            repeat (j, 32-i){ // do this for each of the (32-i) FAs in the ith row
                fa.a[current_row_fa_index+j] = a[j] & b[i]
                fa.b[current_row_fa_index+j] = fa.s[previous_row_fa_index+1+j]
                if (j == 0){ // set the cin input of the leading FA of the ith row to be 0
                    fa.cin[current_row_fa_index+j] = 0
                }
                else{
                    fa.cin[current_row_fa_index+j] = fa.cout[current_row_fa_index+j-1]
                }
            }
            previous_row_fa_index = current_row_fa_index
            current_row_fa_index = current_row_fa_index + 32-i
            mul[i] = fa.s[previous_row_fa_index]

        }

        // 31st row (i.e. the last row) and MSB out, this can be combined with the for-loop above,
        // but separated for teaching purposes
        fa.a[495] = a[0] & b[31]
        fa.b[495] = fa.s[494]
```

```
        fa.cin[495] = 0
        mul[31] = fa.s[495]

    }
}
```

- Taken from course material
- AND gates for multiplication of bits
- Chained adders and AND gates as described
- ‘I’th row has 32 - i adders
- Number of FAs needed = 31 + … + 1 = 496



multiplier.luc

Rotator unit

```
module rotater #(SIZE = 32 : SIZE > 2) (
    input a[SIZE],
    input b[$clog2(SIZE)],
    input alufn[6],
    output shift[SIZE]
) {
    always {
        if (alufn[0]) { // right rotate
            shift = (a >> b) | (a << (SIZE - b))
        }
        else {
            shift = (a << b) | (a >> (SIZE - b))
        }
    }
}
```

- Similar to shifter unit but the bits that are shifted out of the bit-width will wrap around to the other side
- Useful for wrap-around scrolling of graphics (moving obstacles downward)

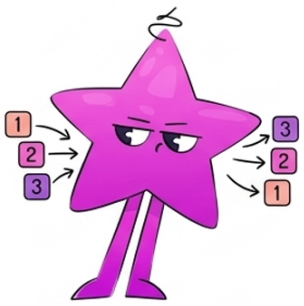
rotater.luc

Reverse unit

```
module reverse #(
    SIZE = 32: SIZE > 0
)(
    input in[SIZE],
    output out[SIZE]
) {
    always {
        repeat(i, 32) {
            out[i] = in[31-i]
        }
    }
}
```

Reverse unit

- Reverses the output of adder or multiplier unit
- ALUFN[1:0] still used for choosing between add/multiply and add/subtract
- Useful for flipping graphics (symmetrically)
- Was used in shifter unit



reverse.luc

Absolute unit

```
module abs #(SIZE = 32 : SIZE > 2)(
    input in[SIZE],
    output out[SIZE]
) {
    sig n
    always {
        n = in[-1] //msb of in
        out = (in ^ SIZEx{n}) + n // if negative number signalled by 1 in msb, then xor all bits of in and add
        // 1
    }
}
```

- Applies absolute to the output of adder or multiplier unit
- ALUFN[1:0] still used for choosing between add/multiply and add/subtract
- Does not work on 0x8000 0000 as it is not representable in positive 2's complement for 32-bit
- Absolute subtraction useful for distance calculations

abs.luc

Testers:

- Manual (fsm_manual.luc)
- Automatic (fsm_eq.luc)
- Selected by the second pin from the left of the Alchitry Au board
- Datapath modules written in the same file as the state machines
    - Does not contain the entire datapath but just enough to simplify alchitry_top

Manual Tester

```
module fsm_manual (
    input clk,  // clock
    input rst,  // reset
    input btn,
    output reg_en[4],
    output seg_indicator[8]
) {
    enum States {
        AL, // A low 16 bits
        AH, // A high 16 bits
        BL, // and so on
        BH,
        ALUFN,
        C // result
    }

    .clk(clk), .rst(rst) {
        dff states[$width(States)](#INIT(States.AL))
    }

    .clk(clk) {
        edge_detector btn_edge(.in(btn), #RISE(1), #FALL(0))
    }

    always {
        states.d = states.q
        reg_en = 0
        seg_indicator = 0
```

Manual Tester

```
        case(states.q) {
            States.AL:
                seg_indicator = 8b11110111 //A.
                reg_en = 4b01
                if (btn_edge.out) {
                    states.d = States.AH
                }
            States.AH:
                seg_indicator = 8b01110111 //A
                reg_en = 4b11
                if (btn_edge.out) {
                    states.d = States.BL
                }
            States.BL:
                seg_indicator = 8b11111100 //B.
                reg_en = 4b101
                if (btn_edge.out) {
                    states.d = States.BH
                }
            States.BH:
                seg_indicator = 8b01111100 //B
                reg_en = 4b111
                if (btn_edge.out) {
                    states.d = States.ALUFN
                }
            States.ALUFN:
                seg_indicator = 8b01010100 //n
                reg_en = 4b010
                if (btn_edge.out) {
                    states.d = States.C
                }
            States.C:
                seg_indicator = 8b00111001 //C
                reg_en = 4b1001
                if (btn_edge.out) {
                    states.d = States.AL
                }
        }
    }
}
```

Manual Tester

```
module datapath_manual (
    input half[16],
    input alufn_in[6],
    input reg_en[4],
    input clk,
    input rst,
    input high_btn,
    input c[32],
    output a[32],
    output b[32],
    output alufn[6],
    output display[16]
) {
    .clk(clk), .rst(rst), #INIT(0) {
```

```
        dff al[16]
        dff ah[16]
        dff bl[16]
        dff bh[16]
        dff alufn_val[6]
    }

    always {
        al.d = al.q
        ah.d = ah.q
        bl.d = bl.q
        bh.d = bh.q
        alufn_val.d = alufn_val.q
        display = 0

        a = c{ah.q, al.q}
        b = c{bh.q, bl.q}
        alufn = alufn_val.q

        case (reg_en) {
            b001:
                al.d = half
            b011:
                ah.d = half
            b101:
                bl.d = half
            b111:
                bh.d = half
```

Manual Tester

```
            b010:
                alufn_val.d = alufn_in
            b1001:
                if (high_btn) {
                    display = c[31:16]
                }
                else {
                    display = c[15:0]
                }
        }
    }

}
```

- State machine to control loading of input data in 16-bit chunks
- Lower 16-bit then upper 16-bit for inputs ‘A’ and ‘B’
- Proceeded by ALUFN 6-bits
- Proceeded by display of lower 16-bits of ALU output
    - Middle button can be pressed to display the upper 16-bits
- Datapath: 16-bit Registers used for inputs of the ALU
- Multiplexer used to enable and load one of these registers at a time

Automatic Tester

```
module fsm_eq #(NUM_TESTS ~ 4: NUM_TESTS > 0)(
    input clk,  // clock
    input rst,  // reset
    input slow_clock,
    input start, // conditioned button
    input equal, // equal signal from the comparator
    output led_indicator[7],
    output error, // error flag factored out of the led_indicator
    output index_value[$clog2(NUM_TESTS)]
) {
    enum States {
        START,
        LOOP,
        CHECK_RESULT,
        WAIT_1,
        WAIT_2,
        HALT
    }

    .clk(clk), .rst(rst) {
        dff states[$width(States)](#INIT(States.START))
        dff index[$clog2(NUM_TESTS)](#INIT(0))
        dff err(#INIT(0)) // hold the error output between states
    }

    .clk(clk) {
        edge_detector rising_edge(.in(slow_clock), #RISE(1), #FALL(0))
        edge_detector rising_edge_start(.in(start), #RISE(1), #FALL(0))
    }
```

Automatic Tester

```
    always {
        index_value = index.q
        states.d = states.q
        index.d = index.q
        error = err.q
        led_indicator = 0

        case(states.q) {
            States.START:
                led_indicator = 7h01
                index.d = 0
                states.d = States.WAIT_1
            States.LOOP:
                led_indicator = 7h41 // bit 6 is the run flag
                if (rising_edge.out) {
                    index.d = index.q + 1
                    states.d = States.WAIT_1
                }
            // states for syncing with rrca delay
            // not really that important with a slow clock but just to be safe
            States.WAIT_1:
                led_indicator = 7h41
                // advance without slow clock because rrca is running on main clock
                states.d = States.WAIT_2
            States.WAIT_2:
                led_indicator = 7h41
                states.d = States.CHECK_RESULT
            States.CHECK_RESULT:
                led_indicator = 7h42
                if (rising_edge.out) {
                    if (~equal) {
                        states.d = States.HALT
                        err.d = 1
                    }
                    else if (~|(index.q ^NUM_TESTS - 1)) {
                        states.d = States.HALT
                    }
                    else {
                        states.d = States.LOOP
                    }
                }
             States.HALT:
                led_indicator = 7h03
                if (rising_edge_start.out) {
                    index.d = 0
                    err.d = 0
                    states.d = States.START
                }
        }
    }
}
```

Automatic Tester

```
module datapath_eq #(NUM_TESTS ~ 4 : NUM_TESTS > 0)(
    input i[$clog2(NUM_TESTS)],
    input clk,
    input rst,
    input high_btn,
    input inv_sw[16],
    input c[32],
    output a[32],
    output b[32],
    output alufn[6],
    output equal,
    output count[8],
    output display[16]
) {
    const TEST_VALUES_ALU = $reverse(
        {
            c{32d0,          32d0,          6h00, c{32d0,          1b1,1b0,1b0}}, // 00 ADD 0+0
            c{32h7FFFFFFF,   32d1,          6h00, c{32h80000000,   1b0,1b1,1b1}}, // 01 ADD pos overflow

            c{32d7,          32d5,          6h01, c{32d2,          1b0,1b0,1b0}}, // 02 SUB 7-5
            c{32h80000000,   32d1,          6h01, c{32h7FFFFFFF,   1b0,1b1,1b0}}, // 03 SUB neg overflow

            c{32d3,          32d4,          6h02, c{32d12,         1b0,1b0,1b0}}, // 04 MUL 3*4
            c{32h80000000,   32d2,          6h02, c{32d0,          1b0,1b0,1b1}}, // 05 MUL 80000000*2

            c{32hF0F0F0F0,   32h0F0F0F0F,   6h18, c{32h00000000,   1b0,1b0,1b1}}, // 06 AND mask clears to zer
            c{32hF0F0F0F0,   32h0F0F0F0F,   6h1E, c{32hFFFFFFFF,   1b0,1b0,1b1}}, // 07 OR fills all bits
            c{32hAAAA5555,   32hFFFF0000,   6h16, c{32h55555555,   1b0,1b0,1b1}}, // 08 XOR pattern
```

Automatic Tester

```
            c{32d5,          32d7,          6h1A, c{32d5,          1b0,1b0,1b0}}, // 09 PASSA 5,7

            c{32d1,          32d3,          6h20, c{32d8,          1b0,1b0,1b0}}, // 10 SHL 1<<3
            c{32d16,         32d1,          6h21, c{32d8,          1b0,1b0,1b0}}, // 11 SHR 16>>1
            c{32h80000000,   32d1,          6h23, c{32hC0000000,   1b0,1b1,1b0}}, // 12 SRA 80000000>>1

            c{32d5,          32d5,          6h33, c{32d1,          1b1,1b0,1b0}}, // 13 CMPEQ 5==5
            c{32d3,          32d7,          6h35, c{32d1,          1b0,1b0,1b1}}, // 14 CMPLT 3<7 true
            c{32d5,          32d5,          6h37, c{32d1,          1b1,1b0,1b0}}, // 15 CMPLE 5<=5

            c{32h80000001,   32d4,          6h24, c{32h00000018,   1b0,1b0,1b1}}, // 16 ROL 1, 4
            c{32d16,         32d1,          6h25, c{32d8,          1b0,1b0,1b0}}, // 17 ROR 16, 1

            c{32h7FFFFFFF,   32d1,          6h28, c{32h00000001,   1b0,1b1,1b1}}, // 18 REVADD overflow
            c{32d7,          32d5,          6h29, c{32h40000000,   1b0,1b0,1b0}}, // 19 REVSUB 7-5
            c{32d3,          32d4,          6h2A, c{32h30000000,   1b0,1b0,1b0}}, // 20 REVMUL 3*4

            c{32h7FFFFFFF,   32d1,          6h2C, c{32h80000000,   1b0,1b1,1b1}}, // 21 ABSADD overflow
            c{32hFFFFFFFF,   32d3,          6h2D, c{32d4,          1b0,1b0,1b1}}, // 22 ABSSUB
            c{32h00000000,   32d3,          6h2D, c{32d3,          1b0,1b0,1b1}}, // 23 ABSSUB
            c{32d3,          32hFFFFFFFE,   6h2E, c{32d6,          1b0,1b0,1b0}}  // 24 ABSMUL 3*-2
        }
    )
    .clk(clk), .rst(rst) {
        dff aa[32]
        dff bb[32]
        dff cc[32]
    }
```

```
always {
    aa.d = TEST_VALUES_ALU[i][104:73] ^ c{16b0, inv_sw} // inv_sw is the dip switches from actual fpga boa
    // if any of the dip switches are flipped, the 'A' input will be wrong and thus output of ALU is wrong
    //triggering the error.
    bb.d = TEST_VALUES_ALU[i][72:41]
    alufn = TEST_VALUES_ALU[i][40:35]

    cc.d = c
    a = aa.q
    b = bb.q

    equal = ~|(cc.q ^ TEST_VALUES_ALU[i][34:3])
    count = i
    display = cc.q[15:0]
    if (high_btn) {
        display = cc.q[31:16]
    }
}
```

- State machine identical to the one in lab 4
- Datapath:
    - Inputs and outputs of ALU put through registers

Automatic Tester

- ‘A’ input connected via XOR gate to an ‘inv_sw’ input which in this case is connected to io_dip[1] and io_dip[0] in alchitry_top
- ‘inv_sw’ toggles lower
- Button input for viewing the upper 16-bits of the result
- ROM for test cases and results

- Multiplexed 7-segment displaying in hexadecimal the test case index

Complete alchitry.top

```
module alchitry_top (
    input clk,              // 100MHz clock
    input rst_n,            // reset button (active low)
    output led[8],          // 8 user controllable LEDs
    input usb_rx,           // USB->Serial input
    output usb_tx,          // USB->Serial output
    output io_led[3][8],    // LEDs on IO Shield
    output io_segment[8],   // 7-segment LEDs on IO Shield
    output io_select[4],    // Digit select on IO Shield
    input io_button[5],     // 5 buttons on IO Shield
    input io_dip[3][8]      // DIP switches on IO Shield
) {

    sig rst                 // reset signal

    alu alu
    .clk(clk) {
        // The reset conditioner is used to synchronize the reset signal to the FPGA
        // clock. This ensures the entire FPGA comes out of reset at the same time.
        reset_conditioner reset_cond
        button_conditioner start_button(#CLK_FREQ($is_sim() ? 1000 : 100000000), .in(io_button[0]))
        button_conditioner high_button(#CLK_FREQ($is_sim() ? 1000 : 100000000), .in(io_button[1]))

        .rst(rst) {
            // Reduce DIV values for simulation so it doesn't take forever
            multi_seven_seg seg (#DIV($is_sim() ? 0 : 16), #DIGITS(4)) // turns on one segment at a time
            // but very quickly, so it looks like the it is a steady number shown on the display

            counter slow_clock(#DIV($is_sim() ? 8 : 27), #SIZE(1)) // fsm to run at a slower speedso we can
            //see the test cases being run
            fsm_manual fm(.btn(start_button.out))
            #NUM_TESTS(25) {
```

Complete alchitry.top

```
                fsm_eq fe(.slow_clock(slow_clock.value), .start(start_button.out))
                datapath_eq de(.high_btn(high_button.out), .c(alu.out))
            }
            .high_btn(high_button.out) {
                datapath_manual dm(.reg_en(fm.reg_en), .c(alu.out))
            }
        }
    }

    always {
        reset_cond.in = ~rst_n  // input raw inverted reset signal
        rst = reset_cond.out    // conditioned reset

        led = c{3b0, io_button} // connect buttons to LEDs
        io_led = 3x{{8h0}}

        usb_tx = usb_rx         // loop serial port

        seg.values = 4x{{4h0}} // assign all 0 first

        dm.half = c{io_dip[1], io_dip[0]}
        dm.alufn_in = io_dip[2][5:0]

        de.i = fe.index_value
        de.inv_sw = c{io_dip[1], io_dip[0]}
        alu.a = de.a
        alu.b = de.b
        alu.alufn = de.alufn
        fe.equal = de.equal
        led = c{fe.error, fe.led_indicator} // error bit is MSB so it will appear at the bottom of the
```

```
        // leds
        seg.values = {4h0, 4h0, de.count[7:4], de.count[3:0]}
        io_led[1] = de.display[15:8] // result of ALU displayed on io_led
        io_led[0] = de.display[7:0]

        io_segment = ~seg.seg // connect segments to the driver
        io_select = ~seg.sel  // connect digit select to the driver

        if (io_dip[2][6]) { // Manual enabled
            alu.a = dm.a
            alu.b = dm.b
            alu.alufn = dm.alufn
            io_led[1] = dm.display[15:8]
            io_led[0] = dm.display[7:0]
            io_segment = ~fm.seg_indicator
            io_select = 4hE
        }
    }
}
```